

3-Statement Financial Model Engine

Complete Technical Overview

- › Framework: Next.js 16 + React 19 + TypeScript 5
- › Engine: Iterative Circular Resolver (100-iteration convergence)
 - › Export: Excel (9 tabs, live formulas) · PDF · CSV · JSON
 - › Analysis: Monte Carlo · Sensitivity · Scenario Comparison
 - › Localization: English / Arabic · Egyptian Market Support

Table of Contents

- 1. Technology Stack
- 2. Project Structure
- 3. Type System (types/)
- 4. Engine Layer (lib/engines/)
- 5. Financial Ratios
- 6. Analysis Tools
- 7. State Management
- 8. Export Layer
- 9. Internationalization & Egyptian Localization
- 10. Utility Functions
- 11. UI Components (21 total)
- 12. Navigation & Tab System
- 13. Data Flow Architecture
- 14. Key Design Decisions
- 15. Known Limitations

1. Technology Stack

Layer	Technology	Version
Framework	Next.js (Turbopack)	16.1.6
Language	TypeScript	5.x
UI	React	19.2.3
State	Zustand (persist)	5.0.11
Charts	Recharts + D3	3.7 / 7.9
Excel Export	ExcelJS	4.4
PDF Export	jsPDF + AutoTable	4.1 / 5.0
Validation	Zod + TypeBox	4.3 / 0.34
CSS	Tailwind CSS	4.x
Math	mathjs	15.1

2. Project Structure

```
3_Statement_Model_Engine/
% % %   a p p /                               #   N e x t . j s   a p p   r o u t e r
%       % % %   p a g e . t s x                 #   M a i n   p a g e   ( t a b   r o u t e r )
%       % % %   g l o b a l s . c s s           #   D e s i g n   s y s t e m   ( 8 . 9   K B )
% % %   t y p e s /                             #   T y p e S c r i p t   i n t e r f a c e s
%       % % %   a s s u m p t i o n s . t s     #   A s s u m p t i o n S e t   ( 6 0 +   f i e l d s )
%       % % %   f i n a n c i a l . t s         #   I S ,   B S ,   C F ,   R a t i o s ,   M o d e l R e s u l t s
%       % % %   h i s t o r i c a l . t s       #   H i s t o r i c a l D a t a I n p u t ,   c o n v e r t e r s
%       % % %   s c e n a r i o . t s           #   S c e n a r i o ,   M o d e l S t a t e ,   M C   t y p e s
% % %   l i b /
%       % % %   e n g i n e s /
%       %       % % %   i n c o m e - s t a t e m e n t . t s   #   R e v e n u e ! '   N e t   I n c o m e
%       %       % % %   b a l a n c e - s h e e t . t s       #   A s s e t s / L i a b i l i t i e s / E q u i
%       %       % % %   c a s h - f l o w . t s               #   I n d i r e c t   m e t h o d   C F
%       %       % % %   c i r c u l a r - r e s o l v e r . t s   #   I t e r a t i v e   s o l v e r
%       %       % % %   i n t e g r a t o r . t s             #   M a s t e r   o r c h e s t r a t o r
%       % % %   e x p o r t /
%       %       % % %   e x c e l . t s                     #   9 - t a b   w o r k b o o k   ( 1 0 3   K B )
%       %       % % %   p d f . t s                         #   M u l t i - p a g e   r e p o r t   ( 4 9   K B )
%       %       % % %   c s v - j s o n . t s               #   C S V   +   J S O N   e x p o r t
%       % % %   s t o r e . t s                             #   Z u s t a n d   ( 1 6   K B ,   1 5 +   a c t i o n s )
%       % % %   r a t i o s . t s                           #   2 3   f i n a n c i a l   r a t i o s
%       % % %   m o n t e - c a r l o . t s                 #   4   d i s t r i b u t i o n s ,   1 0 K   i t e r
%       % % %   s e n s i t i v i t y . t s                 #   1 - w a y   a n d   2 - w a y   a n a l y s i s
%       % % %   s c e n a r i o - m a n a g e r . t s       #   C R U D   +   c o m p a r i s o n
% % %   c o m p o n e n t s /                             #   2 1   R e a c t   U I   c o m p o n e n t s
```

3. Type System

AssumptionSet (types/assumptions.ts) — 60+ fields

Category	Key Fields
Revenue	revenueBase, revenueGrowthRate[]
Margins	cogsPercent[], sgaPercent[], rdPercent[], otherOpexPercent[]
Working Capital	dso[], dio[], dpo[], prepaidPercent[], accruedExpPercent[], deferredRevPercent[]
Non-Current Assets	capexPercent[], depreciationRate[], amortizationAmount[], intangibles, goodwill

Debt	interestRate[], shortTermDebt[], LTD issuance/repayment, currentPortionLTD[]
Equity	commonStock[], APIC[], sharesOutstanding[], SBC, dividendPayout%, equityIssuance[]
Tax / VAT	taxRate[], enableVAT, vatRate, interestIncomeRate[]
Egyptian	buildings, machinery, vehicles, computers, furniture (PP&E asset-class mix)
Model Config	projectionYears, historicalYears, startYear, fiscalYearEnd

Financial Statement Interfaces (types/financial.ts)

Interface	Fields	Purpose
IncomeStatement	26	Revenue !' EPS waterfall + VAT memo
BalanceSheet	30	Full A = L + E with balance check
CashFlowStatement	25	CFO/CFI/CFF + FCF + reconciliation
FinancialRatios	23	Profitability / Liquidity / Leverage / Efficiency
IntegrationChecks	16 booleans	Cross-statement validation
ModelResults	Combined	IS[] + BS[] + CF[] + Ratios[] + Convergence

Historical Data (types/historical.ts)

- HistoricalDataInput — Per-year input (55+ fields), user-editable
- convertToHistoricalInputs() — Bridges per-year !' array format for engine
- buildHistoricalYear() — Auto-computes derived totals + retained earnings plug
- validateHistoricalBalance() — Returns error if Assets != Liabilities + Equity

Scenario & Analysis Types (types/scenario.ts)

- ModelState — Global app state (company info, 15 tabs, undo/redo stacks, errors)
- Scenario — {id, name, type, assumptions, results, timestamps}
- ScenarioType — base | optimistic | conservative | custom
- Distribution — normal | uniform | triangular | lognormal
- MonteCarloResult — with p10/p25/p50/p75/p90 percentile statistics

4. Engine Layer

Income Statement Engine (income-statement.ts)

```

Revenue = previousRevenue × (1 + growthRate)    // ALWAYS chains
COGS = Revenue × cogsPercent
Gross Profit = Revenue - COGS
OpEx = SG&A + R&D + D&A + Other + SBC
EBIT = Gross Profit - Total OpEx
EBITDA = EBIT + Depreciation + Amortization
EBT = EBIT + IntIncome - IntExp + OtherInc
Tax = max(0, EBT × taxRate)
Net Income = EBT - Tax
EPS = NetIncome / SharesOutstanding

```

Also supports VAT memo fields (Egyptian market): revenueInclVAT, revenueExclVAT, vatCollected.

Balance Sheet Engine (balance-sheet.ts)

Item	Formula
Accounts Receivable	Revenue × DSO / 365
Inventory	COGS × DIO / 365
Prepaid Expenses	Revenue × prepaidPercent
Gross PP&E	Prior + CapEx (Revenue × capexPercent)
Accum. Depreciation	Prior + Depreciation
Net PP&E	Gross - Accumulated

Accounts Payable	$\text{COGS} \times \text{DPO} / 365$
Accrued Expenses	$\text{Revenue} \times \text{accruedExpPercent}$
Long-Term Debt	$\text{Prior} + \text{Issuance} - \text{Repayment}$
Retained Earnings	$\text{Prior} + \text{NetIncome} - \text{Dividends}$
APIC	$\text{Prior} + \text{SBC} + \text{EquityIssuance}$
Cash	$\text{PLUG} = \text{Total L+E} - \text{all other assets}$

Cash Flow Engine (cash-flow.ts) — Indirect Method

```

OPERATING: NI + D&A + SBC + DefTax + WC Changes = CFO
INVESTING: -CapEx - Acquisitions + AssetSales = CFI
FINANCING: +DebtIssue - DebtRepay - Dividends + Equity = CFF
Net Change = CFO + CFI + CFF
FCF = CFO - CapEx
Reconciles? = |EndingCash - BS.Cash| < 0.01

```

Circular Resolver (circular-resolver.ts) — THE HEART

Resolves the fundamental circular dependency: Interest! Cash! CF! Net Income! with tolerance = 0.01. Each iteration: compute Depreciation! Interest! IS! BS! convergence.

16 Integration Checks (validateIntegration)

#	Check	Formula
1	Assets Balance	Total Assets = Total L+E
2	Cash Ties	CF Ending Cash = BS Cash
3	Net Income Flows	IS Net Income = CF Net Income
4	PP&E Ties	Net PP&E = Gross - Accum Dep
5	RE Flows	RE = Prior + NI - Dividends
6	Debt Ties	LTD = Prior + Issue - Repay
7	CF Reconciles	Ending = Begin + Net Change
8	WC Ties	WC changes = BS deltas
9-13	Subtotal checks	CA, NCA, CL, NCL, Equity sums
14	IS Waterfall	Revenue - COGS - Gross - NI
15	EBITDA Identity	EBITDA = EBIT + D + A
16	APIC Consistency	APIC = Prior + SBC + Equity

Integrator (integrator.ts) — Master Orchestrator

runFullModel(assumptions, historicalInputs): 1) Build historical IS/BS/CF! 2) For resolveCircularReferences! validateIntegration! 3) Combine historical + projected ModelResults.

5. Financial Ratios (ratios.ts)

Category	Ratios
Profitability	Gross Margin, Operating Margin, Net Margin, ROE, ROA, ROIC
Liquidity	Current Ratio, Quick Ratio, Cash Ratio
Leverage	Debt-to-Equity, Debt-to-Assets, Interest Coverage
Efficiency	Asset Turnover, Inventory Turnover, Receivables Turnover, DSO, DIO, DPO, CCC

Uses average balances for ROE, ROA, and turnover ratios. ROIC = NOPAT / avg invested capital. Interest Coverage

= EBIT / Interest Expense.

6. Analysis Tools

Sensitivity Analysis ([sensitivity.ts](#))

- `oneWaySensitivity()` — Varies one assumption across a range, measures output metric
- `twoWaySensitivity()` — Varies two assumptions simultaneously, produces NxN matrix
- Output metrics: revenue | ebitda | netIncome | eps | fcf | roe

Monte Carlo Simulation ([monte-carlo.ts](#))

- 4 distributions: Normal (Box-Muller), Uniform, Triangular, Lognormal
- Default: 10,000 iterations with revenue growth, COGS%, and interest rate as variables
- Statistics: mean, median, stdDev, p10, p25, p50, p75, p90, min, max

Scenario Manager ([scenario-manager.ts](#))

- 3 pre-defined: Base (moderate growth), Optimistic (high growth + margin expansion), Conservative (slow growth + margin pressure)
- CRUD: `createScenario`, `duplicateScenario`, `updateScenarioAssumption`, `deleteScenario`
- `compareScenarios()` — Side-by-side: Revenue, EBITDA, NI, EPS, ECF, Debt, Cash, ROE

7. State Management ([store.ts](#))

Zustand store with localStorage persistence. 396 lines, 15+ actions.

Action	Description
<code>setCompanyInfo()</code>	Update company metadata (name, ticker, currency, country)
<code>setHistoricalData()</code>	Set per-year data + auto-convert to engine format
<code>updateAssumption(path, val)</code>	Handles nested paths like <code>revenueGrowthRate[0]</code>
<code>addScenario()</code> / <code>delete</code> / <code>dup</code>	Scenario CRUD operations
<code>calculateModel()</code>	Runs <code>runFullModel()</code> for active scenario
<code>calculateAllScenarios()</code>	Runs model for every scenario in parallel
<code>undo()</code> / <code>redo()</code>	Assumption-level undo/redo (unlimited stack)
<code>setCountryPreset()</code>	Applies US / Egyptian / Custom tax and depreciation defaults
<code>resetToDefaults()</code>	Full state reset to demo data

8. Export Layer

Excel Export ([excel.ts](#)) — 2,029 lines, 103 KB

Generates a 9-tab Excel workbook with 100% LIVE FORMULAS (no hard-coded values):

Tab	Content
Summary Dashboard	Scenario comparison matrix (all scenarios side-by-side)
Assumptions	40+ editable rows that drive all formulas
Income Statement	Revenue EPS with formulas referencing Assumptions
Balance Sheet	Full A=L+E with Cash as plug formula
Cash Flow Statement	Indirect method, formulas reference IS and BS
Ratios	Profitability, Liquidity, Efficiency — all formulas
Working Capital	DSO/DIO/DPO + NWC schedule
Depreciation Schedule	Gross PP&E rollforward + accumulated depreciation

- Historical vs Projected visual styling (blue tint for historical, border separator)
- Back-computed assumptions for historical periods (DSO, DIO, DPO from actual data)
- Cash plug: Total L+E " AR " Inventory " Prepaid " OCA " Total NCA

- Balance check: `IF(ABS(Assets " L+E) < 1, " Balanced", " Imbalanced")`
- CF reconciliation: `IF(ABS(EndingCash " BS.Cash) < 0.01, " ", " ")`

PDF Export (pdf.ts) — 49 KB

- Multi-page report: Cover, Income Statement, Balance Sheet, Cash Flow, Ratios
- Supports Egyptian locale with bilingual labels

CSV / JSON Export (csv-json.ts)

- Plain data export in CSV and JSON formats for external tools

9. Internationalization & Egyptian Localization

Bilingual Labels (i18n/labels.ts)

80+ financial line items with English/Arabic label pairs covering IS, BS, CF, Ratios, and Egyptian-specific terms.

```
revenue: { en: 'Revenue', ar: 'إيرادات' }
accountsReceivable: { en: 'Accounts Receivable', ar: 'مبيعات عملاء' }
vatRate: { en: 'VAT Rate', ar: 'معدل ضريبة القيمة المضافة' }
```

Egyptian Depreciation (schedules/egyptian-depreciation.ts)

Asset Class	Arabic	Min Rate	Max Rate	Typical
Buildings	بنايات	2%	5%	4%
Machinery	آلات	7%	10%	8%
Vehicles	سيارات	12.5%	25%	20%
Computers	حواسيب	25%	33%	33%
Furniture	أثاث	10%	20%	15%

Egyptian tax defaults: Corporate tax 22.5%, VAT 14%, Dividend withholding 10%. Fiscal year presets: Calendar (Jan-Dec), Egyptian Govt (Jul-Jun), Custom.

10. Utility Functions (utils.ts)

Function	Purpose
formatCurrency(val, cur, compact)	Multi-currency, ALWAYS English numerals (never Arabic i-`)
formatPercent(val, decimals)	Percentage formatting (e.g., "15.2%")
formatNumber(val, decimals)	Number formatting with en-US locale
formatEPS(val, currency)	EPS with currency symbol
colorForValue(val)	Green (>0) / Red (<0) / Neutral CSS color

Supported currencies: USD (\$), EGP (££), EUR (€), GBP (£), SAR (SR), AED (AED).

11. UI Components (21 total)

Core Pages

Component	Size	Purpose
Sidebar.tsx	11.7 KB	Navigation (15 tabs), Calculate, Export, Dark mode
Dashboard.tsx	14.6 KB	KPI cards + Scenario comparison table with deltas
ModelPage.tsx	7 KB	Assumptions editor — 60+ fields in sections
ValidationPage.tsx	7.8 KB	80 integration checks (16 × 5 years) pass/fail

Financial Statements

Component	Size	Purpose
IncomeStatementPage.tsx	6.5 KB	Full IS with historical (blue) vs projected columns
BalanceSheetPage.tsx	7 KB	Assets, Liabilities, Equity + balance check
CashFlowPage.tsx	7.1 KB	CFO/CFI/CFF + reconciliation indicator

Schedules

Component	Size	Purpose
WorkingCapitalPage.tsx	10.9 KB	DSO/DIO/DPO, NWC build, NWC % Revenue
DepreciationPage.tsx	8.2 KB	Gross PP&E rollforward, Egyptian asset-class sliders
DebtSchedulePage.tsx	9.1 KB	Debt issuance/repayment, interest calculation

Analysis & Settings

Component	Size	Purpose
SensitivityPage.tsx	6 KB	Tornado chart + data table
MonteCarloPage.tsx	9.4 KB	Histogram + percentile statistics
ScenariosPage.tsx	5.7 KB	Scenario list, create/duplicate/delete
ScenarioComparisonModal	12 KB	Full side-by-side comparison modal
CompanySettings.tsx	3.2 KB	Company name, ticker, currency, country
EgyptianSettings.tsx	8.7 KB	VAT, tax rates, fiscal year, depreciation mix
HistoricalImportPage.tsx	12.3 KB	CSV import with column mapping
RevenueChart.tsx	2.4 KB	Revenue bar chart (Recharts)
MarginChart.tsx	1.5 KB	Gross/Operating/Net margin line chart

12. Navigation & Tab System

The app uses a 15-tab system controlled by the Zustand store's activeTab:

```
dashboard !' income !' balance !' cashflow !' model !'
working-capital !' depreciation !' debt-schedule !'
scenarios !' sensitivity !' montecarlo !'
import !' historicaldata !' validation !' company-settings
```

13. Data Flow Architecture

INPUT: Historical Data (2-3 years) + Assumptions (60+ fields) + Scenario Select

ENGINE: integrator.runFullModel() !' Build Historical IS/BS/CF !' For each project (iterate up to 100x) !' IS !' BS !' CF loop !' validateIntegration (16 checks)

OUTPUT: ModelResults !' UI Components + Excel (9 tabs, live formulas) + PDF (r

ANALYSIS: ModelResults !' Sensitivity Analysis + Monte Carlo (10K iterations) +

14. Key Design Decisions

- Cash as Plug — Cash is the balancing item (Total L+E " all other assets). $A =$
- Iterative Circular Resolution — Interest !" Cash !" CF !" NI. Solved via 100-itera
- Revenue Chain — ALWAYS chains previousRevenue $\times (1 + \text{growthRate})$. No separate projection base.
- Average Balances for Interest — Interest expense = avg(begin, end) debt $\times$ rate. Same for income.
- Back-Computed Historical Assumptions — For Excel, historical DSO/DIO/DPO are back-computed so all cells can be formulas.
- Egyptian Localization — Full support: 22.5% corp tax, 14% VAT, 5 asset-class depreciation, bilingual labels.
- Zustand + localStorage — State persists across browser refreshes.

15. Known Limitations

- No multi-segment revenue breakdown
- No DCF / terminal value / WACC valuation module
- No automated import from financial data APIs (SEC, Bloomberg, etc.)
- Single currency only — no FX conversion or multi-currency BS
- No user authentication or multi-user collaboration
- Monte Carlo runs synchronously (may freeze UI for large iteration counts)
- Historical data limited to manual entry or CSV import